Migración del catálogo de apicultores a MySQL con PDO: CRUD de productos y socios con consultas preparadas

Ronald Salvatierra Flores

Facultad de Ingeniería de Sistemas, Universidad Privada Domingo Savio

SIS-0151 Tecnología Web II

M.Sc. Ing. Jimmy Nataniel Requena Llorentty

15 de octubre de 2026

Actividad 2 — Informe técnico final

Repositorio: https://github.com/RonSalvet/tw2_actividad02.git

Resumen

Este informe documenta la migración de la aplicación web de la Asociación de Apicultores Flor de la Chiquitanía, que almacenaba sus datos en archivos JSON, a una base de datos MySQL 8.4 accedida con PDO. Se diseñó el modelo entidad-relación (socios 1:N productos), un script SQL con claves foráneas, restricciones y datos de ejemplo, y una conexión configurada mediante variables de entorno fuera del directorio público. Sobre esa base se construyeron los repositorios ProductoRepositorio y SocioRepositorio con operaciones de crear, leer, actualizar y eliminar mediante consultas preparadas, validación en el servidor y paginación con filtros y orden de lista permitida. La resistencia a la inyección SQL se demostró con una comparación entre una consulta vulnerable de carácter didáctico y el repositorio real, y con 15 comprobaciones automáticas que incluyen una prueba de mutación. El código, las guías y los verificadores de los laboratorios 5 al 9 se publican en un repositorio de GitHub.

Palabras clave: MySQL, PDO, consultas preparadas, inyección SQL, patrón repositorio, PHP

Migración del catálogo de apicultores a MySQL con PDO: CRUD de productos y socios con consultas preparadas

Introducción

La asociación Flor de la Chiquitanía ofrece miel, derivados y cera producidos por sus socios. Hasta el laboratorio 7 la aplicación guardaba el catálogo en archivos JSON, una solución que no escala: no garantiza integridad entre productos y socios, no permite consultas eficientes y obliga a leer todo el archivo en cada petición. La Actividad 2 de Tecnología Web II pide migrar la aplicación a MySQL y completar el CRUD de productos y socios con PDO y consultas preparadas, sin ninguna consulta vulnerable a inyección SQL, que es la defensa principal contra ese tipo de ataque (MITRE, s. f.; OWASP Foundation, s. f.).

Este informe describe el diseño de la base de datos (laboratorio 8), la capa de repositorios con validación y paginación (laboratorio 9), la ampliación que hizo falta para que los socios también tengan un CRUD completo y la demostración de que la aplicación resiste la inyección SQL. El código completo está en el repositorio del proyecto: https://github.com/RonSalvet/tw2_actividad02.git.

Objetivos

Objetivo general

Migrar la aplicación a MySQL y completar el CRUD de productos y socios con PDO y consultas preparadas, sin consultas vulnerables a inyección SQL.

Objetivos específicos

- Modelar la base de datos (entidad-relación) y escribir el script SQL con tablas, claves foráneas, restricciones y datos de ejemplo.
- Configurar la conexión PDO fuera del código público, con las claves en variables de entorno.
- Implementar los repositorios de productos y de socios con CRUD, validación y paginación.
- Demostrar con ataques reales que la aplicación resiste la inyección SQL.
- Publicar los laboratorios y la actividad en un repositorio de GitHub.

Entorno y método de trabajo

Se trabajó en Windows 10 con Docker Desktop y Docker Compose (Docker, Inc., s. f.), con dos servicios: php (imagen php:8.4-cli con la extensión pdo_mysql y Composer) y mysql (imagen mysql:8.4). El autoload sigue el estándar PSR-4 (espacio de nombres Apicultores\ en src/). La base de datos se crea sola la primera vez que arranca el volumen, porque los archivos de la carpeta sql/ se montan en docker-entrypoint-initdb.d. Cada laboratorio se comprobó con un verificador propio del curso, que levanta el servidor integrado de PHP, hace peticiones HTTP reales y muestra el resultado de cada regla.

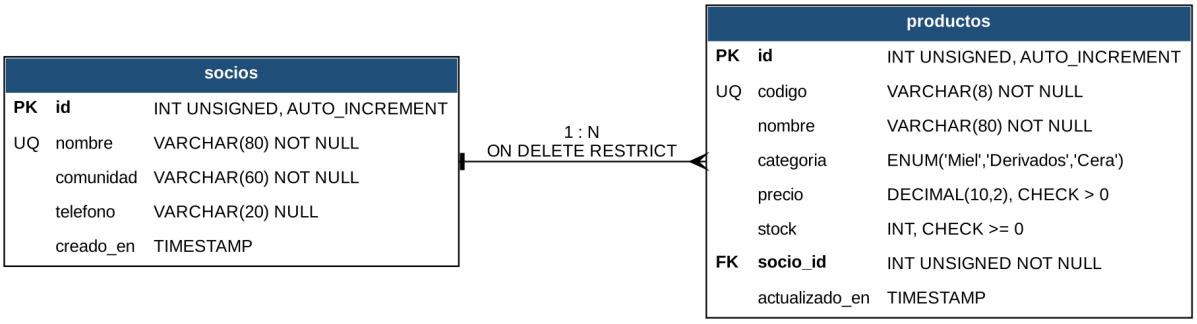
Laboratorio 8: modelo de datos y conexión PDO

Modelo entidad-relación

El dominio tiene dos entidades: socios y productos. Un socio produce muchos productos y cada producto pertenece a un único socio, es decir, una relación uno a muchos (1:N) que se materializa con la clave foránea productos.socio_id (Figura 1). Se eligió ON DELETE RESTRICT para que no sea posible borrar un socio que todavía tiene productos.

Figura 1

Modelo entidad-relación de la base de datos apicultores



Nota. Elaboración propia a partir del script sql/01-esquema.sql. PK = clave primaria, FK = clave foránea, UQ = restricción UNIQUE.

Script SQL

El script sql/01-esquema.sql crea las tablas con el motor InnoDB y el juego de caracteres utf8mb4. Las reglas de negocio se declararon en la propia base para que ningún camino de acceso pueda saltárselas (Tabla 1). El precio se guarda como DECIMAL(10,2) y no como

FLOAT, porque los tipos de coma flotante no representan exactamente valores como 0,1 y los totales monetarios dejarían de cuadrar (Oracle Corporation, s. f.). Los archivos 02-datos.sql y 03-mas-datos.sql cargan los datos de ejemplo: 8 socios y 36 productos.

Tabla 1

Restricciones declaradas en la base de datos

Restricción	Tabla	Qué garantiza
PRIMARY KEY (id)	socios, productos	Cada fila tiene un identificador único.
UNIQUE uq_socios_nombre	socios	No hay dos socios con el mismo nombre.
UNIQUE uq_productos_codigo	productos	No hay dos productos con el mismo código.
CHECK precio > 0	productos	El precio siempre es positivo.
CHECK stock >= 0	productos	El stock nunca es negativo.
FOREIGN KEY socio_id → socios(id) ON DELETE RESTRICT	productos	Todo producto tiene un socio existente; no se borra un socio con productos.

Nota. Fuente: sql/01-esquema.sql del repositorio.

Conexión PDO fuera del código público

La configuración de la conexión está en config/bd.php, una carpeta que queda fuera de public/, el único directorio que el servidor web expone. El archivo no contiene secretos: lee el servidor, el puerto, la base, el usuario y la clave de variables de entorno (BD_HOST, BD_PUERTO, BD_NOMBRE, BD_USUARIO, BD_CLAVE) que define docker-compose.yml. Así las credenciales se pueden cambiar por entorno sin tocar el código. La clase Conexion crea la conexión PDO con tres opciones (Tabla 2) (PHP Group, s. f.-a).

Tabla 2

Opciones de PDO usadas en la conexión

Opción	Valor	Efecto
ATTR_ERRMODE	ERRMODE_EXCEPTION	Los errores de SQL son excepciones y no pasan inadvertidos.
ATTR_DEFAULT_FETCH_MODE	FETCH_ASSOC	Cada fila llega como arreglo asociativo.
ATTR_EMULATE_PREPARES	false	Consultas preparadas reales: MySQL recibe primero la estructura y después los datos.

Nota. Fuente: src/BaseDatos/Conexion.php del repositorio.

Cuando la base no responde, index.php captura la excepción, escribe el detalle en datos/errores.log con una referencia aleatoria y muestra al usuario una página 500 amable que solo contiene esa referencia: no aparecen usuario, clave, servidor ni puerto.

Laboratorio 9: repositorios, validación y paginación

Patrón repositorio

Un repositorio es una interfaz que describe cómo se guardan y consultan los objetos, de modo que el resto de la aplicación no conoce el almacenamiento (Fowler, 2002). ProductoRepositorio es el contrato; ProductoRepositorioMysql lo implementa con PDO y ProductoRepositorioMemoria lo implementa sin base de datos, y el verificador comprueba que ambos responden igual. El método privado hidratar() convierte cada fila de MySQL en un objeto Producto con su Socio. Para la Actividad 2 se aplicó el mismo patrón a los socios: SocioRepositorio ahora es una interfaz (todos, porId, buscar, existeNombre, contarProductos, guardar, eliminar) y SocioRepositorioMysql es su implementación. Los controladores reciben las interfaces, de modo que cambiar de motor de base de datos solo exige una nueva implementación.

Paginación, filtros y orden seguro

El listado de productos no trae las 36 filas, sino una página de diez. Primero ejecuta SELECT COUNT(*) con los mismos filtros para conocer el total y luego la consulta con LIMIT :limite OFFSET :desde, enlazando ambos valores con bindValue y PDO::PARAM_INT, porque MySQL exige enteros en esas posiciones (PHP Group, s. f.-b). La clase Pagina calcula el total de páginas con ceil() y un mínimo de uno: 36 productos de diez en diez son cuatro páginas. El filtro por categoría agrega la condición p.categoria = :categoria. El orden usa una lista permitida (nombre, precio, precio_desc, stock): un nombre de columna no puede ir en un marcador, de modo que a SQL solo llega un fragmento escrito por el programador y nunca el texto recibido. El controlador normaliza los parámetros de la URL: un orden desconocido vuelve al orden por nombre, una página inválida (abc, -5) vuelve a la primera y una página mayor que la última lleva a la última.

Validación

Toda entrada se valida en el servidor antes de llegar al repositorio. ValidadorProducto exige un código con formato AAA-999 y único, un nombre de 3 a 80 caracteres, una categoría de la lista, un precio entre 0,01 y 10.000, un stock entero no negativo y un socio existente. Para los socios se escribió ValidadorSocio: nombre de 3 a 80 caracteres y único, comunidad de 2 a 60 y teléfono opcional de 7 a 12 dígitos. Si hay errores se responde 422 y el formulario vuelve a mostrarse con los mensajes y los datos escritos. Además, la base de datos repite las reglas más importantes (Tabla 1) como última línea de defensa: por ejemplo, el repositorio traduce el error 1062 de MySQL (entrada duplicada) a una excepción Conflicto, porque dos personas podrían registrar el mismo código a la vez y superar ambas la validación.

CRUD completo de productos y socios

La aplicación expone el CRUD de las dos entidades con un único front controller (public/index.php) y un enrutador (Tabla 3). Las operaciones que modifican datos usan POST y responden con una redirección 303 (patrón Post/Redirect/Get), de modo que recargar la página no reenvía el formulario. El borrado de un socio con productos se rechaza con 409, tanto por una comprobación del repositorio que traduce el error 1451 de la clave foránea como por la propia restricción de la base.

Tabla 3

Rutas del CRUD de productos y de socios

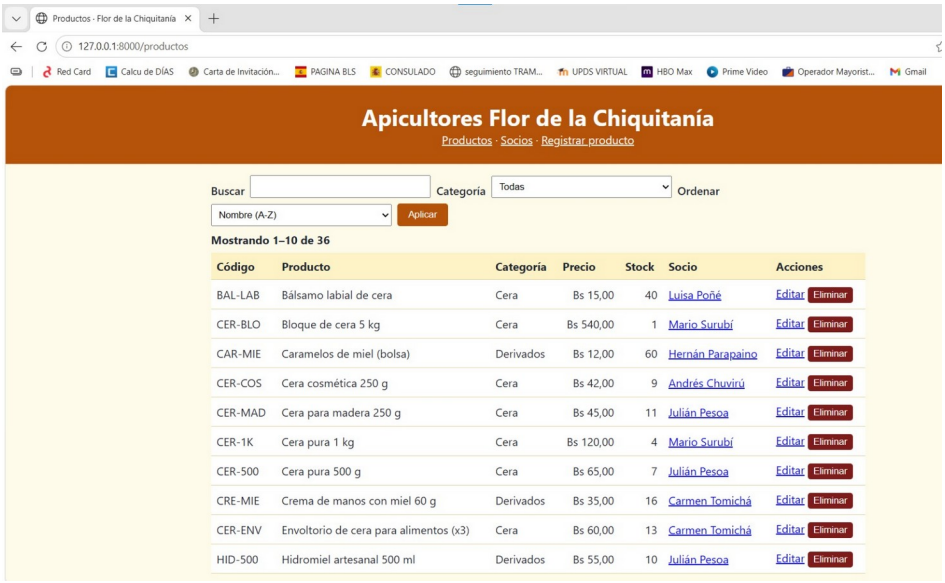
Método y ruta	Acción	Respuesta
GET /productos	Listar con búsqueda, filtro, orden y paginación	200
GET /productos/nuevo · POST /productos	Formulario · registrar producto	200 · 303 / 422
GET /productos/{codigo}/editar · POST /productos/{codigo}	Formulario · actualizar producto	200 · 303 / 422 / 404
POST /productos/{codigo}/eliminar	Eliminar producto	303 / 404
GET /socios	Listar socios con búsqueda y paginación	200
GET /socios/nuevo · POST /socios	Formulario · registrar socio	200 · 303 / 422
GET /socios/{id} · GET /socios/{id}/editar	Ficha del socio con sus productos · formulario	200 / 404

Método y ruta	Acción	Respuesta
POST /socios/{id}	Actualizar socio	303 / 422 / 404
POST /socios/{id}/eliminar	Eliminar socio (rechaza si tiene productos)	303 / 409 / 404

Nota. Los métodos que no corresponden a una ruta existente responden 405 con la cabecera Allow. Fuente: public/index.php.

Figura 2

Lista paginada de productos (primera página)



Nota. Captura propia de http://127.0.0.1:8000/productos: «Mostrando 1–10 de 36».

Seguridad: resistencia a la inyección SQL

Defensas

Ninguna consulta de los repositorios MySQL incorpora datos del usuario en el texto SQL. Cada vector de entrada tiene su defensa (Tabla 4), siguiendo las recomendaciones de OWASP Foundation (s. f.): consultas parametrizadas para los valores, lista permitida para los nombres de columna y validación de tipos para los enteros.

Tabla 4

Vectores de entrada y defensa aplicada

Entrada del usuario	Dónde llega	Defensa
Texto de búsqueda (q)	LIKE en productos y socios	Marcadores :patron y :patron2; los comodines % _ \ se escapan con addcslashes.

Entrada del usuario	Dónde llega	Defensa
Categoría	Condición p.categoria	Se convierte a un enum (tryFrom) y se enlaza como marcador.
Orden	ORDER BY	Lista permitida; clave desconocida → orden por nombre.
Página	LIMIT / OFFSET	filter_var entero ≥ 1 y bindValue con PARAM_INT.
Código o id en la ruta	WHERE codigo = ? / id = ?	Marcador ?; el id debe ser solo dígitos.
Campos de formularios	INSERT / UPDATE	Validación en el servidor y marcadores con nombre.

Nota. Fuente: src/Repositorios y src/Controladores del repositorio.

Demostración del ataque

El script demo_inyeccion.php ejecuta las mismas entradas maliciosas contra una función vulnerable escrita solo para la demostración (concatena el texto en el SQL y solo hace SELECT) y contra los repositorios reales. La entrada zzz' OR 1=1 -- - convierte la condición en verdadera y la versión vulnerable devuelve las 36 filas; la carga con UNION SELECT extrae el nombre, la comunidad y el teléfono de los socios, datos que la búsqueda de productos nunca debería mostrar; una comilla suelta rompe la consulta con el error 1064. Con consultas preparadas las tres entradas se comparan como texto y devuelven 0 resultados, y la tabla productos permanece intacta (Figura 3).

Figura 3

Salida de demo_inyeccion.php: consulta vulnerable frente a consulta preparada

```
DEMOSTRACIÓN DE INYECCIÓN SQL — productos en la tabla: 36
------------------------------------------------------------
Condición siempre verdadera
  entrada del usuario : zzz' OR 1=1 -- -
  [VULNERABLE] SQL ejecutado: SELECT codigo, nombre, categoria FROM productos WHERE nombre LIKE '%zzz' OR
1=1 -- -%'
  [VULNERABLE] resultado: 36 filas → ejemplo: CER-MAD | Cera para madera 250 g | Cera
  [SEGURO    ] consulta preparada: WHERE (p.nombre LIKE :patron OR p.codigo LIKE :patron2) → 0 productos, 0
socios
------------------------------------------------------------
UNION: robar socios y teléfonos
  entrada del usuario : zzz' UNION SELECT nombre, comunidad, COALESCE(telefono, '-') FROM socios -- -
  [VULNERABLE] SQL ejecutado: SELECT codigo, nombre, categoria FROM productos WHERE nombre LIKE '%zzz' UNION
SELECT nombre, comunidad, COALESCE(telefono, '-') FROM socios -- -%'
  [VULNERABLE] resultado: 8 filas → ejemplo: Andrés Chuvirú | Roboré | -
  [SEGURO    ] consulta preparada: WHERE (p.nombre LIKE :patron OR p.codigo LIKE :patron2) → 0 productos, 0
socios
```

```
------------------------------------------------------------
Comilla suelta (rompe el SQL)
  entrada del usuario : '
  [VULNERABLE] SQL ejecutado: SELECT codigo, nombre, categoria FROM productos WHERE nombre LIKE '%'%'
  [VULNERABLE] resultado: ERROR de MySQL 1064 (la entrada rompió la consulta)
  [SEGURO    ] consulta preparada: WHERE (p.nombre LIKE :patron OR p.codigo LIKE :patron2) → 0 productos, 0
socios
------------------------------------------------------------
Tabla productos después de todos los ataques: 36 (antes: 36) → INTACTA
```

Nota. Salida de php demo_inyeccion.php. Cada bloque muestra el SQL que se ejecuta realmente en la versión vulnerable y el resultado de la consulta preparada.

Pruebas automáticas

El verificador verificar_actividad2.php ejecuta 15 comprobaciones: cinco del repositorio de socios (consulta, búsqueda paginada, alta y edición, nombre repetido y borrado con la restricción de clave foránea), cinco del CRUD por HTTP (cuatro de socios y una de productos) y cinco de seguridad. Las de seguridad envían nueve cargas de ataque (entre ellas condiciones siempre verdaderas, UNION SELECT, DROP TABLE, SLEEP y comodines) por la búsqueda de productos, la búsqueda de socios, la categoría, el orden, la página, la ruta y los formularios, y exigen que no haya errores 500 ni mensajes SQL en la respuesta, que el número de filas de las tablas no cambie y que un texto de ataque guardado en un formulario se almacene literalmente y se muestre escapado. La quinta comprobación de seguridad analiza el código fuente de los repositorios y falla si algún texto SQL interpola variables.

Para comprobar que las pruebas detectan una aplicación vulnerable se hizo una prueba de mutación: se reemplazó la búsqueda de socios por una versión que concatena el texto, y el verificador pasó de 15 a 13 comprobaciones superadas, con fallos precisamente en la prueba de inyección de la búsqueda de socios y en el análisis del código fuente.

Limitaciones

La defensa contra la inyección SQL no sustituye otras medidas que quedan para actividades posteriores del proyecto: autenticación, protección contra CSRF en los formularios y un usuario de base de datos con privilegios mínimos (hoy el usuario apicultores tiene todos los

permisos sobre su base). Las claves de docker-compose.yml son de desarrollo y están a la vista; en producción deben proveerse como secretos del entorno.

Resultados de verificación

La Tabla 5 resume los resultados de los verificadores de cada laboratorio. Los laboratorios 5, 6, 7 y 9 fueron ejecutados por el estudiante en Docker con las capturas que se adjuntan a cada hoja de trabajo (Figura 4 muestra la del laboratorio 9).

Tabla 5

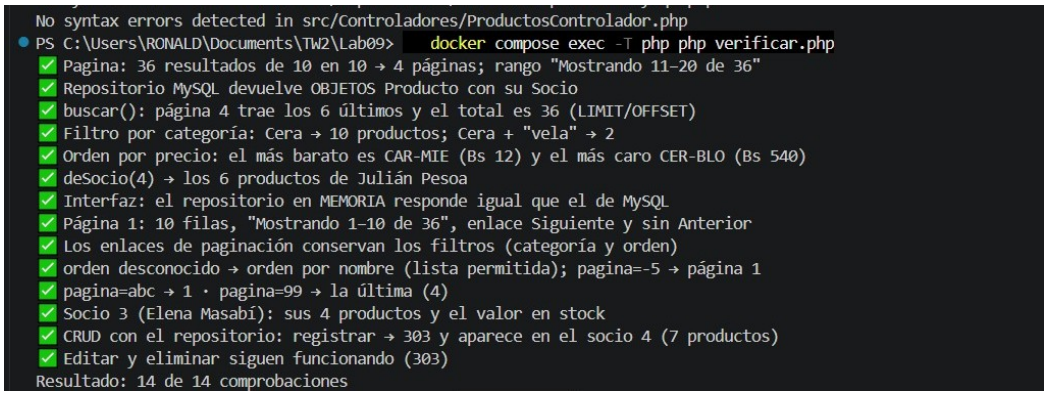
Resultados de los verificadores

Laboratorio / actividad	Tema	Resultado
Laboratorio 5	Objetos y clases en PHP	14 de 14
Laboratorio 6	Herencia, interfaces y Composer	14 de 14
Laboratorio 7	Excepciones, front controller y plantillas	15 de 15
Laboratorio 8	MySQL 8.4 con PDO	15 de 15 (primera versión: 14 de 15)
Laboratorio 9	Repositorio, paginación y filtros	14 de 14
Actividad 2	CRUD de socios y resistencia a inyección SQL	15 de 15

Nota. Fuente: salida de php verificar.php de cada laboratorio. En el laboratorio 8 la primera versión del verificador esperaba el código SQLSTATE 23000 para la violación de CHECK, y MySQL 8.4 informa HY000 (error 3819); se corrigió la comprobación para aceptar cualquier rechazo de la base. Los 15 de 15 del laboratorio 8 corregido y de la Actividad 2 se obtuvieron en la preparación del proyecto con PHP 8.3 y MariaDB 10.11; las capturas con MySQL 8.4 en Docker se incorporan al repositorio como evidencia.

Figura 4

Verificador del laboratorio 9



Nota. Captura propia: 14 de 14 comprobaciones del laboratorio 9.

Dificultades y soluciones

Tabla 6

Problemas encontrados y cómo se resolvieron

Problema	Causa	Solución
Docker Desktop no arrancaba	WSL 2 incompleto	Reinstalar y actualizar WSL y reiniciar el equipo.
No se encontraba el archivo compose	Comando ejecutado en una carpeta sin docker-compose.yml	Entrar a la carpeta inicio del laboratorio y comprobar con Test-Path.
El servicio php «is not running»	Contenedores detenidos	docker compose up -d --build --wait y revisar docker compose ps.
No se conectaba a la base del laboratorio 9	El compose creaba la base «apicultor» y PHP usaba «apicultores»	Se unificó el nombre en MYSQL_DATABASE.
El verificador del laboratorio 8 daba 14 de 15	MySQL informa HY000 en CHECK, no 23000	Se ajustó la comprobación.
404 en /productos'	Una comilla pegada al final de la dirección	Abrir /productos sin la comilla.

Nota. Fuente: registro de trabajo del estudiante.

Uso de inteligencia artificial

Durante el proyecto se utilizó el modelo de lenguaje Claude (Anthropic, 2026) como asistente: revisión de los códigos de partida, redacción de código y de verificadores, explicación de errores y borradores de las hojas de trabajo. Cada uso se registra en la bitácora de IA que acompaña a este informe (Bitacora_IA_Actividad02), con la solicitud, el aporte y la verificación realizada. El código generado se comprobó siempre ejecutando los verificadores, y las respuestas de las hojas de trabajo se contrastaron con las capturas y salidas reales.

Conclusiones

La aplicación quedó migrada a MySQL con un modelo 1:N protegido por claves foráneas y restricciones, con la conexión configurada por variables de entorno fuera de public/. Los repositorios de productos y de socios implementan el CRUD completo con consultas preparadas, validación y paginación, y los controladores dependen de interfaces, lo que permite cambiar de motor de base de datos sin tocarlos. La demostración y las 15 comprobaciones automáticas

muestran que la aplicación resiste la inyección SQL en todos sus puntos de entrada, y la prueba de mutación confirma que esas pruebas detectarían una regresión. Quedan como trabajo futuro la autenticación, la protección CSRF y un usuario de base de datos con privilegios mínimos.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Anthropic. (2026). *Claude* (Sonnet 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai

Docker, Inc. (s. f.). *Docker Compose overview*. https://docs.docker.com/compose/

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

MITRE. (s. f.). *CWE-89: Improper neutralization of special elements used in an SQL command ('SQL injection')*. https://cwe.mitre.org/data/definitions/89.html

Oracle Corporation. (s. f.). *MySQL 8.4 reference manual*. https://dev.mysql.com/doc/refman/8.4/en/

OWASP Foundation. (s. f.). *SQL injection prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

PHP Group. (s. f.-a). *PDO::prepare*. En *PHP manual*. https://www.php.net/manual/en/pdo.prepare.php

PHP Group. (s. f.-b). *PDOStatement::bindValue*. En *PHP manual*. https://www.php.net/manual/en/pdostatement.bindvalue.php

Requena Llorentty, J. N. (2026). *Guías didácticas de los laboratorios de Tecnología Web II (laboratorios 5 al 9)* [Material de curso]. Universidad Privada Domingo Savio.

Anexo A

Estructura del repositorio y cómo ejecutar el proyecto

Repositorio: https://github.com/RonSalvet/tw2_actividad02.git

```
tw2_actividad02/
├── actividad02/              Aplicación final (MySQL + PDO + repositorios + CRUD de socios)
│   ├── config/bd.php         Conexión por variables de entorno (fuera de public/)
│   ├── sql/                  Esquema, datos y más datos (36 productos, 8 socios)
│   ├── src/                  Repositorios, controladores, validadores, modelo
│   ├── verificar.php         Verificador del laboratorio 9 (14 comprobaciones)
│   ├── verificar_actividad2.php   Verificador de la actividad (15 comprobaciones)
│   └── demo_inyeccion.php    Demostración de inyección SQL
├── laboratorios/             lab05 … lab09 (carpetas inicio de cada laboratorio)
└── docs/                     Informe técnico, bitácora de IA, guías completadas, diagrama ER
```

Comandos (PowerShell, dentro de actividad02):

```
docker compose up -d --build --wait --wait-timeout 180
docker compose exec php composer dump-autoload
docker compose exec -T php php verificar.php              # 14 de 14
docker compose exec -T php php verificar_actividad2.php   # 15 de 15
docker compose exec -T php php demo_inyeccion.php
# Sitio: http://127.0.0.1:8000/productos   y   http://127.0.0.1:8000/socios
```